



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

软件体系结构

第14章 体系结构的设计实现

赵庆玲

南京理工大学计算机科学与工程学院

软件体系结构设计和实现



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 软件体系结构是软件系统质量的核心
- 成功的软件体系结构应具有以下品质
 - (1) 良好的模块化
 - (2) 适应功能需求的变化，适应技术的变化
 - (3) 对系统的动态运行有良好的规划
 - (4) 对数据的良好规划
 - (5) 明确、灵活的部署规划



软件体系结构设计和实现

从需求分析到体系结构设计

从软件体系结构到详细设计

体系结构设计原则

软件体系结构设计面临的主要威胁及其对策

从需求分析到体系结构设计 (1)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 需求分析和软件体系结构设计是软件全生命周期中两个关键活动。
- 需求分析主要考虑问题域和系统责任，其目的是得到一个正确、一致并且无二义的需求规约，作为后续开发、验证及系统演化的基础。
- 软件体系结构设计是一个体系结构的定义、文档编写、维护、改进和验证正确实现的活动。它的目的是将系统的高层结构显式地表达出来，在较高的抽象层次上为后续的开发活动提供“蓝图”。

从需求分析到体系结构设计 (2)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 需求到软件体系结构的映射是一个既复杂又细致的工作，对相同的需求采用不同的映射机制会得到不同风格的体系结构。
- **Paul Grunbacher**认为有下面两个概念能够有效指导如何从需求向体系结构进行映射
 - (1) 体系结构风格 (architecture style)：它可以捕获结构、行为和交互模式等；
 - (2) 特定领域的软件体系结构 (DSSA, domain-specific software architecture)：关联模型和它的应用领域。

- 在从用户需求获取软件体系结构的过程中，有两件事情必须要做的：
- （1）探索如何用软件体系结构的概念和描述手段在较高抽象层次上刻画问题空间（用户需求）的软件需求，获得软件需求规约说明书；
- （2）探讨如何从软件需求规约说明书自动或半自动地变换到软件体系结构设计。

软件体系结构对需求的影响 (1)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 用传统的方法产生需求规约，不考虑软件体系结构概念和原则，则在软件体系结构设计阶段建立需求规约与体系结构的映射将相对困难。
- 若把体系结构概念引入需求分析阶段，有助于保证需求规约、系统设计之间的可追踪性和一致性，有效保持软件质量。
- 将软件体系结构概念和原则引入需求分析，也可以让我们获得更有结构性和可重用的需求规约。

- 梅宏等提出的一种面向软件体系结构的需求工程方法步骤
 - (1) 确定问题空间范围。
 - (2) 确定系统用户及他们的职责。
 - (3) 从用户角度探索整体外部行为，产生系统的顶层模型；
 - (4) 确定组件和连接件。
 - (5) 规约组件和连接件。
 - (6) 建立面向软件体系结构的高层需求规约。所得到的需求规约包括组件的需求规约、连接件的需求规约及约束需求规约等。
 - (7) 检查面向软件体系结构需求规约，包括完整性、一致性和死锁等。

- 从软件需求到软件体系结构存在的难点
 - （1）软件需求是频繁获取的非正规的自然语言，而软件体系结构规约常常是一种正式的语言。
 - （2）系统属性中描述的非功能性需求通常很难在体系结构模型中形成规约。
 - （3）需要以迭代和同步演化的方式进行软件需求理解和软件体系结构开发。
 - （4）在从软件需求映射到软件体系结构的过程中，保持一致性和可追溯性很难，且复杂程度很高。
 - （5）大规模系统必须满足数以千计的需求，从而导致很难确定和细化包含这些需求的体系结构相关信息。
 - （6）软件需求和软件体系结构需要满足不同的利益相关者的需求，很难在这些不同利益中找正确的平衡点。

- 把软件需求模型转换为软件体系结构模型的代表性方法：
 - (1) 面向目标和场景的软件体系结构设计方法
 - (2) 基于APL (architecture prescription language) 软件体系结构设计方法
 - (3) 基于规则的体系结构决策诱导框架的软件体系结构设计方法
 - (4) 基于全局分析的体系结构形成方法
 - (5) 面向模式的体系结构形成方法
 - (6) 需求与体系结构协同演化方法
 - (7) 基于CBSP的体系结构形成方法
 - (8) 基于WinWin模型与CBSP结合的体系结构形成方法
 - (9) 面向特征的映射和转换方法

(1) 面向目标和场景的软件体系结构设计方法



- Lin Liu等提出了一种面向目标的需求语言GRL(Goal-oriented Requirement Language)和一种面向场景的体系结构符号UCM(Use Case Maps), 支持面向目标和代理的建模和论证, 并指导体系结构设计过程。
 - GRL是一种支持目标和代理目标建模、推导需求的语言, 特别是处理非功能需求。
 - UCM提供一种场景可视化符号, 描述和推导系统中大粒度行为模式。
- 该建模方法旨在对需求进行启发、求精和实现, 直到得到一个满意的体系结构设计方案。

(1) 面向目标和场景的软件体系结构设计方法



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 步骤:
 - (1) 建立面向目标的需求模型。
 - (2) 建立用例图模型。
 - (3) 在用例图中划分部件, 得到一个参考的体系结构设计方案。
 - (4) 在需求模型中对目标进行求精细化并评估其非功能目标, 如果满足则转入步骤6。
 - (5) 若设计无法满足系统的非功能性目标, 则分析找出新的目标和需求, 寻求其他的解决方案。再返回步骤4
 - (6) 考虑其他的可行方案, 再利用系统的非功能目标进行权衡, 找出最佳的体系结构设计方案。

（2）基于APL软件体系结构设计方法



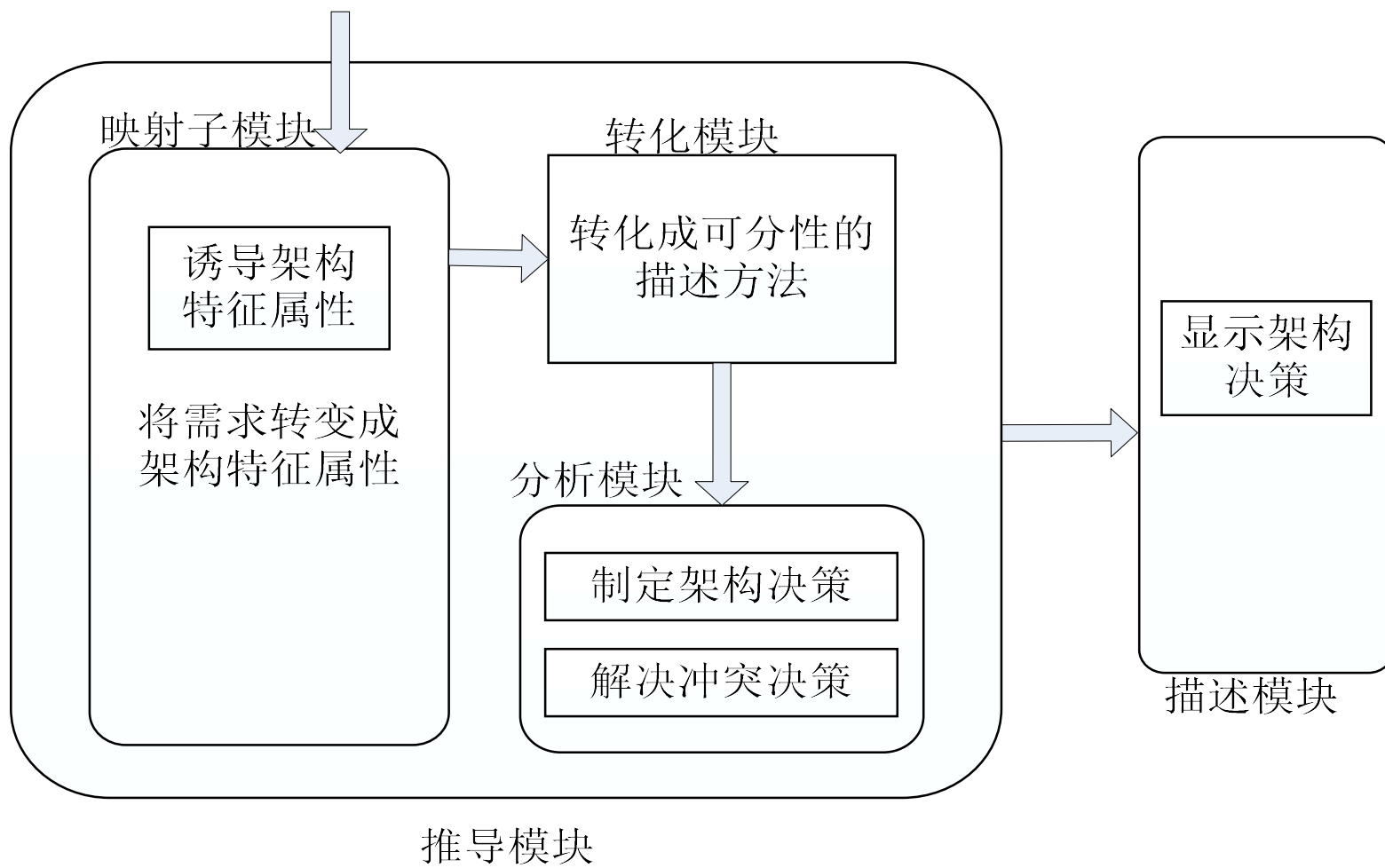
- Brandozzi和Perry介绍了一种体系结构规约语言（APL, architecture prescription language）用于指定体系结构及其组件在应用领域的描述。
- 体系结构规约通过限制体系结构元素（流程、数据、连接件）、交互关系、约束关系展现系统结构空间。
- 步骤：
 - （1）将需求转变成KAOS（knowledge acquisition in automated specification）需求规约
 - （2）KAOS需求规约到软件系统体系结构规范
 - （3）建立体系结构细化树。

(3) 基于规则的体系结构决策 诱导框架



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

Liu wenqian和Steve Easterbrook提出了一个基于规则的体系结构决策诱导框架 (ADEF , architectural decision elicitation framework)



(3) 基于规则的体系结构决策 诱导框架



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

•步骤:

- (1) 借助映射子模块对体系结构特征属性进行诱导。
- (2) 转化成可分析的描述方法。
- (3) 借助分析模块提供自动化推导，制定体系结构决策以及解决冲突决策。
- (4) 显示体系结构决策，并更新以前的结果，然后反复进行生成另外的需求规约。

（4）基于全局分析的体系结构 形成方法设计



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- Hofmeister Christine等人提出了一种更为**通用**的方法，通过“全局分析”实现从需求模型到软件体系结构模型的转化。
 - 其中全局分析包括：
 - （1）分析影响因素；
 - （2）定义问题和制定策略
- 该方法通过影响因素表、问题卡等记录全局分析过程并维护一致性。

（4）基于全局分析的体系结构 形成方法设计



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

步骤：

（1）检查产品需求、使用情况、系统需求及交互，确保理解环境、用户及与该系统交互的其他系统的接口。

（2）分析产品、技术及组织因素，并为这3类因素生成因素表。

（3）识别问题并开发相关策略以解决这些问题。

（4）使用全局分析中生成的策略指导决策，并改进系统的应变能力。

(5) 面向模式的体系结构形成方法



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

MS Rajasree等人提出一种面向模式的软件开发方法。通过这种方法，应用程序框架可以通过先前确定的模式知识来感知，对应的功能需求可进一步围绕这个基本结构进行演化。

步骤：

- (1) 基于整体计算和交互模型，建立应用程序的一个全局性软件体系结构。
- (2) 细化这个软件体系结构，并进行设计。

（6）需求与体系结构协同演化方法：



Paola Inverardi等人提出了一种需求与体系结构协同演化方法，并讨论了模型间一致性检查方法。

步骤：

（1）基于UML的开发过程，通过用例和交互图在需求层面捕获协同策略。

（2）从UML图到体系结构模型。分析步骤1建立的交互图，了解体系结构组件的动态交互关系，通过体系结构语言表述系统行为。

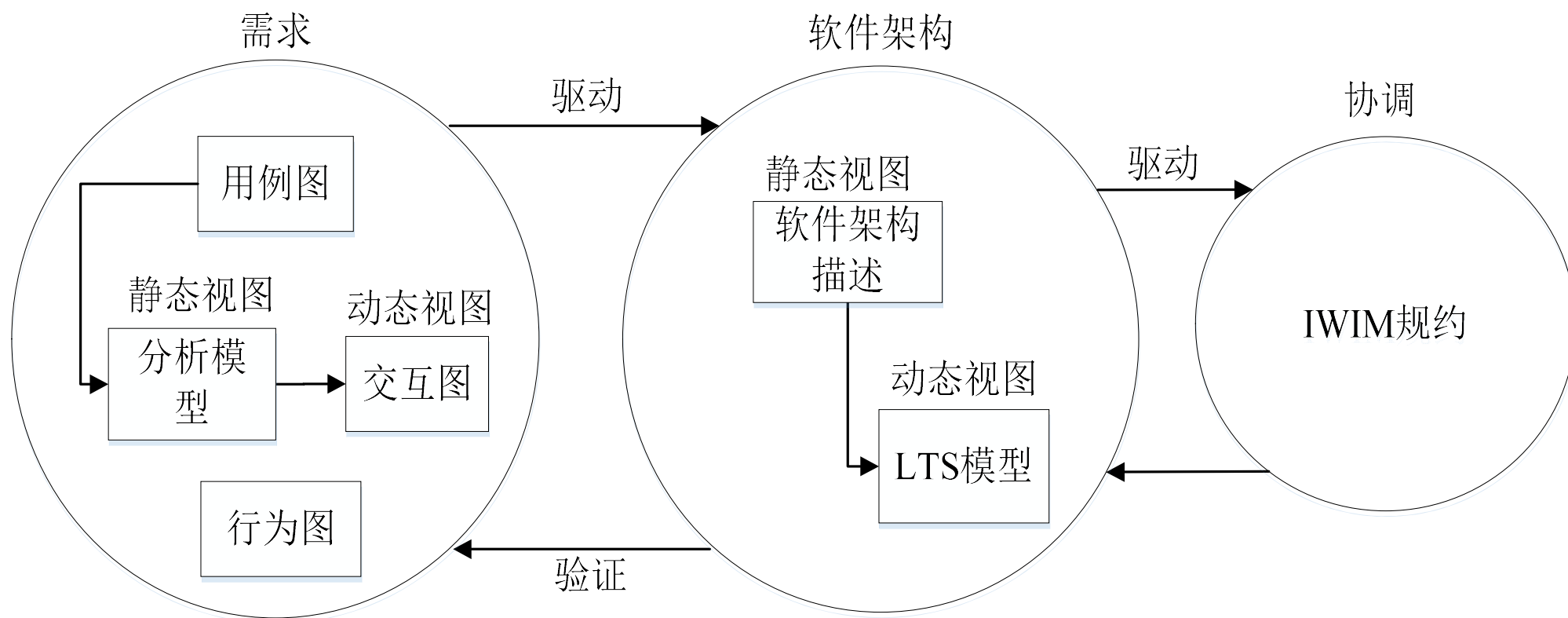
（3）验证软件体系结构动态模型的交互关系。

（4）从软件体系结构模型到IWIM（Idealized Worker Idealized Manager）模型。

(6) 需求与体系结构协同演化方法:



这种方法应用于电信业务和远程医疗保健系统（TRMCS），并得到了很好的发挥。



（7）基于CBSP的体系结构形成方法



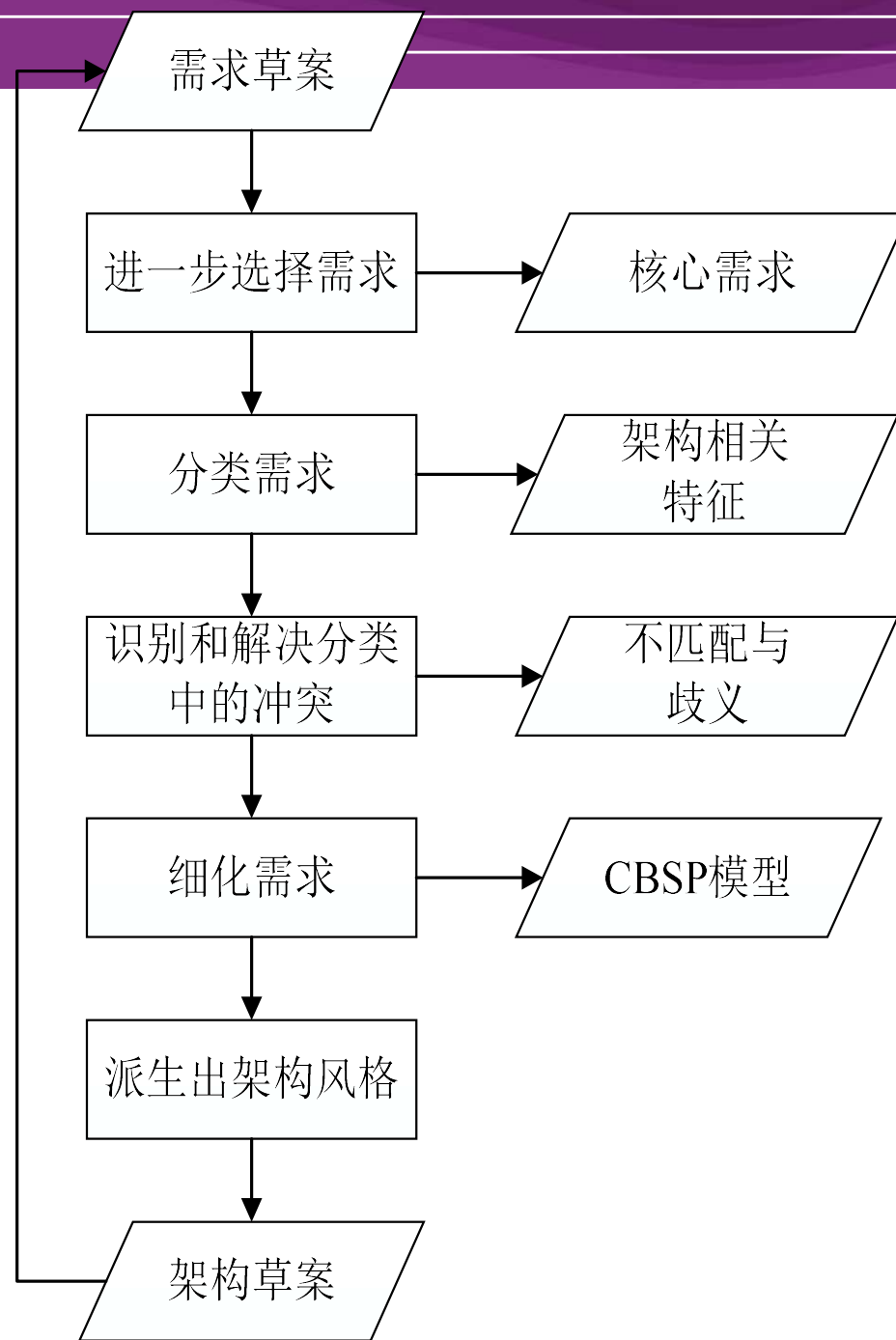
南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- Paul Grunbacher等人提出了一种保持软件需求与体系结构一致性的组件-总线-系统-特性方法（CBSP, Component-Bus-System-Property）方法。
- 步骤：
 - （1）首先定义CBSP分类法
 - （2）进一步选择需求
 - （3）分类需求
 - （4）识别和解决分类中的冲突
 - （5）细化需求
 - （6）派生体系结构风格

(7) 基于CBSP的体系结构形成方法



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



(8) 基于WinWin模型与CBSP结合的体系结构形成方法



- Nenad Medvidovic等人首先开发了WinWin模型用于协助需求协商。
- WinWin定义了一个模型用于指导协商过程：利益相关者的目标被表述为胜利条件，通过捕获已知的约束、问题、冲突进行分析并提供解决方案，如果开发人员与利益相关者达成共识，则方案将被确定。
- 步骤：
 - (1) 用WinWin模型对软件需求进行建模。
 - (2) 用CBSP模型建立需求到体系结构的模型连接器。
 - (3) 对软件体系结构进行建模，根据开发情况，选择满足条件的最优软件体系结构风格。

(9) 面向特征的映射和转换方法



- 刘冬云和梅宏探讨了一种面向特征的映射和转换方法，通过在需求工程和体系结构设计中引入“面向特征”风范，改善了需求模型与软件体系结构之间的映射关系和可追踪性。
- 该方法可适合于迭代的、增量的或演化的开发范型。
- 步骤：
 - (1) 面向特征的需求建模。
 - (2) 概念体系结构建模。
 - (3) 逻辑体系结构建模。
 - (4) 部署体系结构建模。



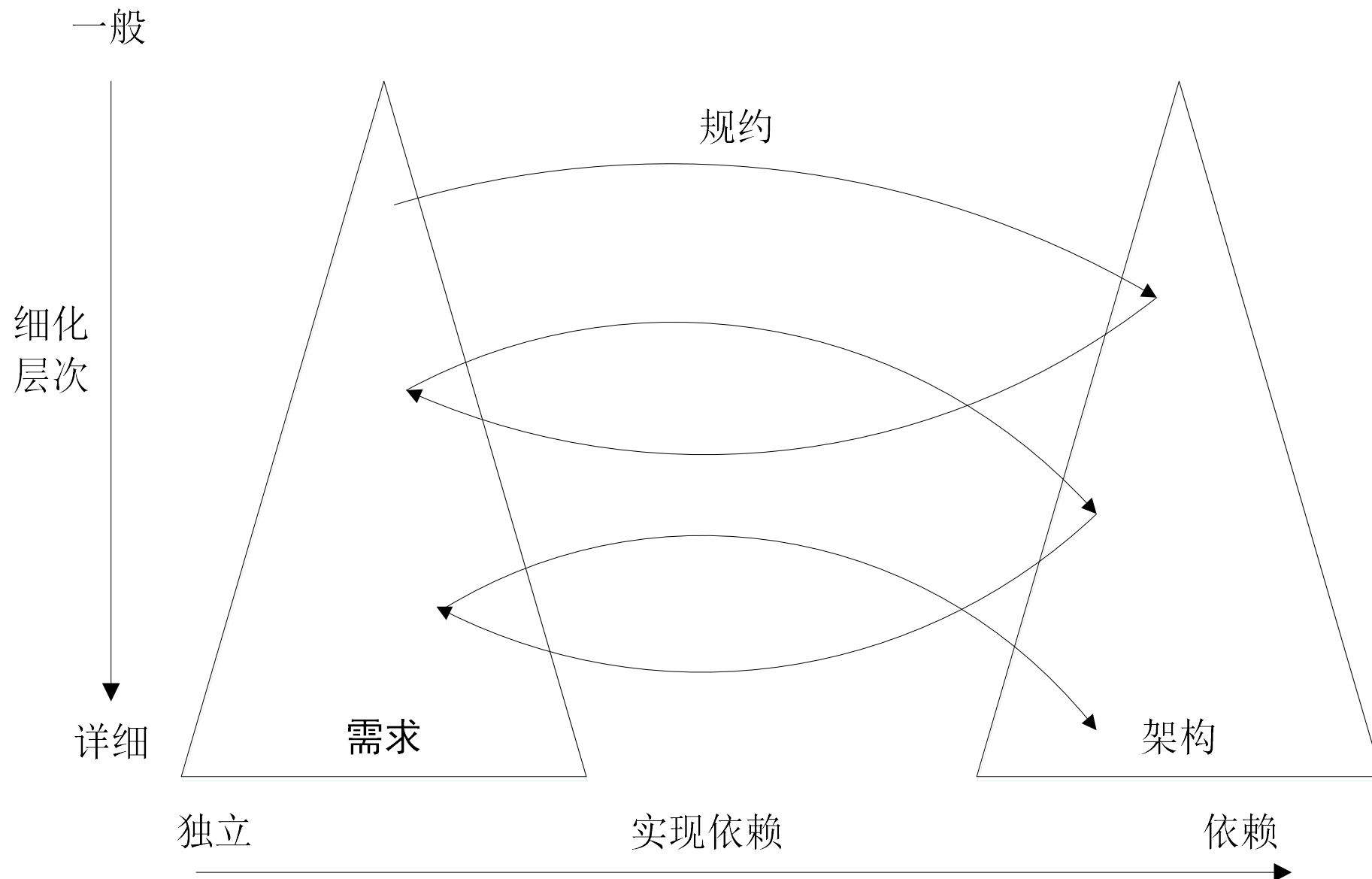
需求与体系结构的协同演化

- 软件需求和软件体系结构两者是相辅相成的关系，一方面软件需求影响软件体系结构设计，另一方面软件体系结构帮助需求分析的明确和细化。
- 需求与体系结构的互相影响可以看成是一个螺旋的过程，也是一个双峰的过程。
 - 双峰模型强调软件需求和软件体系结构的平等性，它是简化版的螺旋模型，在发展需求和体系结构规约同时，继续从解决方案的结构和规约中分离问题的结构和规约，在一个反复的过程中，产生更详细的需求规约和设计规约，最终把交织在软件开发过程中的设计规约和需求规约分离开来。

需求与体系结构的协同演化



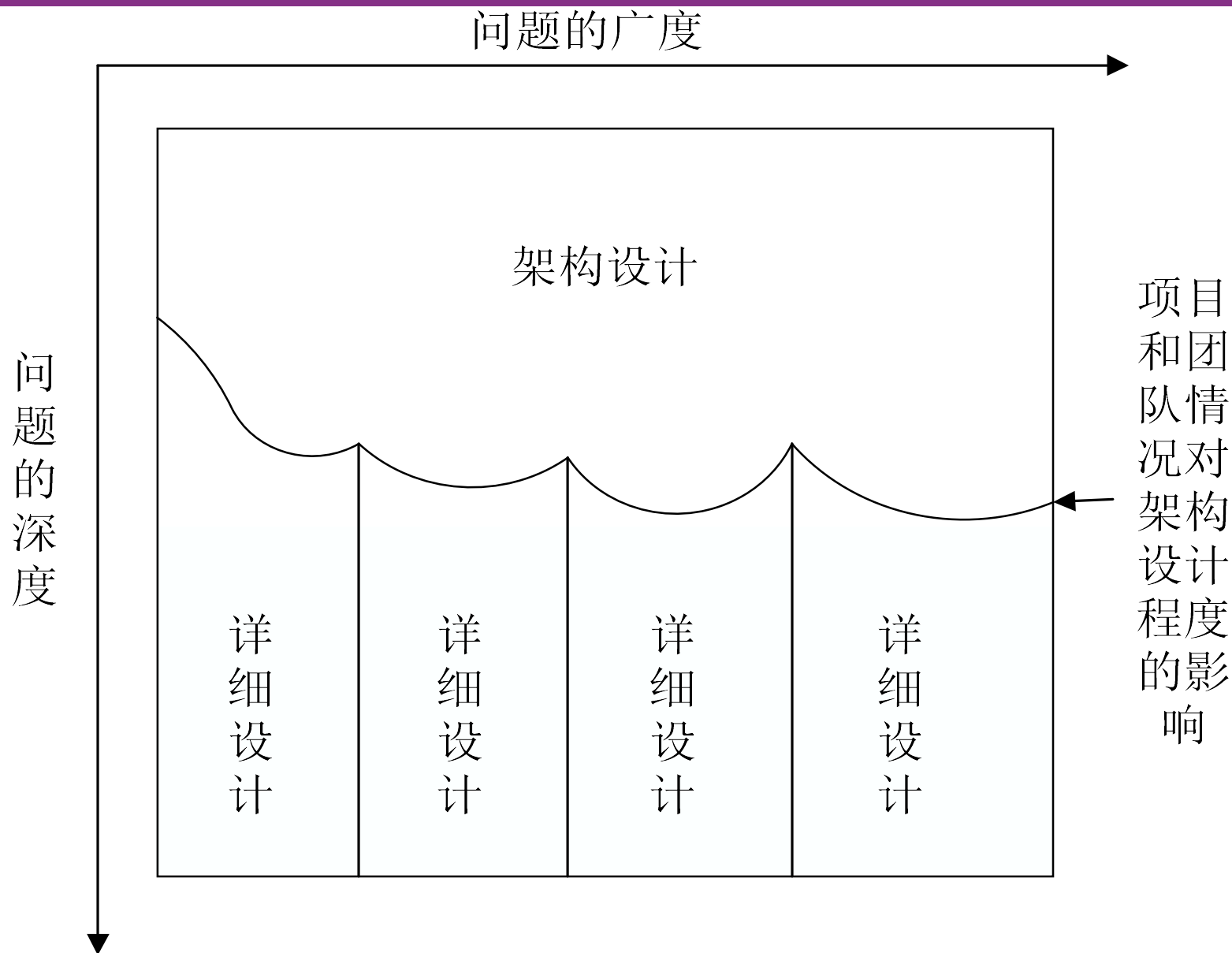
南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



从软件体系结构到详细设计



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



详细设计对软件体系结构的影响（1）



- 详细设计主要集中在体系结构表达式的细化，选择详细的数据结构和算法。
- 具体的说，详细设计阶段就是确定如何具体实现所需的系统，得到一个接近源代码的软件表示。
- 为了促进从体系结构向详细设计的转化，可以在软件体系结构设计中引入详细设计的概念，即在软件体系结构描述语言中引入与实现相关的元素。
 - 如：把Java语言扩展到支持软件体系结构描述的ArchJava；在ADL中引入数据类型和函数等支持详细设计

详细设计对软件体系结构的影响（2）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 如：ArchJava
 - Jonathan Aldrich等人提出，一种新型的软件体系结构描述语言ArchJava
 - 在Java语言中增加了组件、连接件、端口等建模元素用于描述软件体系结构模型。
 - 它将软件体系结构与实现完美的统一起来，确保实现符合体系结构的限制，支持体系结构和实现共同开发。

详细设计对软件体系结构的影响 (3)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 再如：研究者在**ADL**中引入数据类型和函数等支持详细设计
 - **Nenad Medvidovic**等人将面向对象的类型系统引入到**C2**中，并可以在设计阶段通过**OO** 的类、子类型化等概念来规约软件体系结构建模元素。
 - **Dashofy**等人提出的**xADL2**中，允许软件体系结构设计人员定义与平台或语言(如**CORBA**, **Java**等)相关的数据类型、函数声明等

从软件体系结构映射详细设计(1)



- 从设计阶段的软件体系结构模型向代码的转换过程，是将设计阶段的软件体系结构模型逐步精化的过程。
- 软件体系结构映射到详细设计时经常出现的问题：
 - (1) 缺失重要体系结构视图，片面强调功能需求。
 - (2) 不够深入，停留在概念体系结构层面，遗漏了全局性设计决策。
 - (3) 名不副实的分层体系结构，缺失层次之间的交互接口和交互机制，只进行职责划分。
 - (4) 在某些方面过度设计。

从软件体系结构映射详细设计（2）



- 可能的解决方法有：
 - （1）对于缺失重要体系结构视图问题，可以针对遗漏的体系结构视图进行设计。
 - （2）对于不够深入问题，需要将设计决策细化到和技术相关的层面。
 - （3）对于名不副实的分层体系结构问题，需要步步深入，明确各层之间的交互接口和交互机制。
 - （4）虽然我们必须要考虑到系统的扩展性，可维护性等，但切忌过度设计。

从软件体系结构映射详细设计（3）



- 从体系结构描述直接到语言的映射方法：
 - 目前的解决方案或者将高层软件体系结构模型直接映射成为程序代码，或者经过一系列中间模型的转换，渐进地映射到程序代码。
 - 如：C2 体系结构描述语言允许将软件体系结构设计的建模元素映射到面向对象程序设计语言代码，并提供了支持C++和Java的程序库
 - 从ADL向程序代码的映射需要考虑：建立从软件体系结构建模元素到目标语言元素的映射关系、确保映射过程中的语义正确性、提供自动化的转换工具或环境等问题。
 - 通过直接映射将**ADL**转化成为程序语言往往只能生成较为简单的程序代码框架。

从软件体系结构映射详细设计（4）



- 基于模型驱动软件体系结构（MDA, model driven architecture）的映射方法
 - MDA区分了三类模型：
 - 计算独立模型（CIM, Computation Independent Model）
 - 平台独立模型（PIM, Platform Independent Model）
 - 平台特定模型（PSM, Platform Specific Model）

从软件体系结构映射详细设计（5）



- MDA开发步骤
 - （1）用计算独立模型**CIM** 捕获需求；
 - （2）创建平台独立模型**PIM**；
 - （3）将**PIM**转化成为一个或多个平台特定模型**PSM**，并加入平台特定的规则和代码；
 - （4）将**PSM** 转化为代码等。

- 为了更好地发挥软件体系结构在系统实现阶段的指导与交流作用，研究者们提出了若干针对实现阶段的软件体系结构视图
 - **Hofmeister Christine**提出的四种视图：概念视图、模块视图、执行视图、代码视图。
 - 执行视图描述了模块如何映射到运行时平台（**runtime platform**）所提供的元素，以及这些又如何映射到硬件体系结构。
 - 代码视图描述了实现系统的软件是如何被组织的。

- 软件体系结构设计原则有一般设计原则和关键设计原则两类
- 一般原则
 - 包含商业原则、数据原则、应用程序原则、技术原则等；
- 关键设计原则
 - 关注分离点、单一职责原则、最少知识原则等。

体系结构设计的基本原则 (1)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 商业原则：
 - 企业利益最大化
 - 信息管理，人人有责
 - 事务持续性
 - 使用通用软件
 - 守法
 - IT责任
 - 知识产权保护

体系结构设计的基本原则（2）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 数据原则：

- 数据资产
- 数据共享
- 数据访问
- 数据托管
- 使用常用词汇、有数据定义
- 数据安全



- 应用程序原则：
 - 技术独立性
 - 易用性
- 技术原则：
 - 需求变化
 - 响应变更管理
 - 控制技术多样性
 - 互操作性

体系结构设计的关键原则 (1)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 关注分离点
 - 将程序分解为相对独立的、功能重叠部分尽可能少的一个个模块，达到高内聚和低耦合的目的。
- 单一职责原则
 - 每个组件或模块应负责一个特定特征或功能，或者内聚功能的集合。
- 最少知识原则(迪米特法则，LoD)
 - 一个组件或对象应该对其他组件或对象的内部细节尽可能少的了解。

体系结构设计的关键原则 (2)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 不重复自身原则 (DRY):
 - 只须在一个地方指定意图，例如，对于程序设计，特定功能只能在一个组件内实现，且该功能不能复制到任何其他组件内。
- 尽量减小前期设计
 - 只做必要的设计。

软件体系结构设计面临的主要威胁及其对策（1）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 几个面临威胁的方面：
 - 被忽略的重要非功能需求
 - 频繁变化的需求
 - 考虑不全面的体系结构设计
 - 不及时的体系结构验证
 - 较高的创造性的体系结构比重
 - 体系结构的低可执行性

软件体系结构设计面临的主要威胁及其对策（2）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- （1）被忽略的重要非功能需求及其对策
 - 从开发的角度来看，功能需求的缺陷是可以通过编程级的努力补回，而最终提供的软件系统能否达到非功能需求是无法通过编程级的努力达到的，必须重构体系结构。
 - 对策：全面认识需求
 - 软件体系结构强调的是整体，而整体性的设计决策必须给予对需求的全面认识。分层、分类把需求整理清楚，并进行合适的权衡和取舍。

软件体系结构设计面临的主要威胁及其对策（3）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- （2）频繁变化的需求及其对策：
 - 任何需求变更都可能意味着时间和金钱的消耗，并且大量需求变更之后程序可能变得混乱，势必引起Bug增多，影响产品质量。
 - 对策：关键需求决定体系结构
 - 在软件体系结构设计中必须接受变化的普遍存在，采用以关键需求决定体系结构的策略可以有效应对频繁变化的需求。

软件体系结构设计面临的主要威胁及其对策（4）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- （3）考虑不全面的体系结构设计：
 - 软件系统越来越复杂，软件体系结构也越来越复杂；另外，软件体系结构还应该能够反映各方涉众不同角度的关注。
 - 对策：多视图探寻体系结构
 - 采用分而治之的策略，基于多视图的体系结构设计方法使在设计过程中无需对所有方面同时进行考虑，而是可以按关注点每次只围绕少数概念和技术展开。

软件体系结构设计面临的主要威胁及其对策（5）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- （4）不及时的体系结构验证及其对策
 - 体系结构设计的合理性直接影响到软件系统最终是否能够成功，因此若未对软件体系结构合理性进行及早的验证，将为软件开发过程带来巨大的风险。
 - 对策：尽早验证体系结构
 - 应真正对早期体系结构进行编码实现，而不仅仅是评审。要对原型进行测试，重点要确保早期体系结构考虑的质量属性是否能够达到预期。

软件体系结构设计面临的主要威胁及其对策（6）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- （5）较高的创造性的体系结构比重及其对策：
 - 有时，可能理解已有体系结构所耗费的时间比重新设计一个新的体系结构更多，在这样的情景下，设计新体系结构貌似是合理的决策。
 - 对策：合理分配经验体系结构与创新体系结构比重
 - 为了得到成功的软件体系结构，需要权衡确定经验体系结构与创新体系结构比重的分配，不仅要考虑软件开发时间，也要考虑创新体系结构可能引入的风险。调查显示，80%经验体系结构加20%创新体系结构是比较合理的分配比重

软件体系结构设计面临的主要威胁及其对策（7）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- （6）体系结构的低可执行性：
 - 软件体系结构是从较高层次对软件的抽象，但体系结构设计本质是为了保障软件开发过程的顺利进行，所以体系结构设计并非“好的就是成功的”，而是“适合的才是成功的”。
 - 对策：验证体系结构的可执行性
 - 进行体系结构可执行性验证能够对体系结构设计过程产生约束效果，从而避免过度设计给软件项目开发带来的风险。